

MODULE BIG DATA · M2 · ÉPREUVE E05

Big Data

Architecture de données & analytique

Fondamentaux avant de lancer le projet fil rouge —
l'Entrepôt de Données de Santé du CHU

ELT & médaille

ClickHouse

RGPD santé

BI / dashboards

De quoi on va parler

1

Le monde de la Big Data

Les 5V · pourquoi maintenant

2

Les métiers & la stack

Data eng / analyst / scientist ·
Postgres → warehouse

3

Structurer & transformer

ETL → ELT · batch / flux ·
médaillon

4

Modéliser la donnée

Schéma en étoile · fait ou
dimension

5

L'entrepôt

Ligne vs colonne ·
partitionnement · ClickHouse

6

Gouvernance & conformité

RGPD & données de santé

7

Le projet & l'épreuve

Le sujet EDS · l'épreuve E05

L'OBJECTIF DE LA SEMAINE

Construire un vrai EDS, de bout en bout

Le CHU veut un **Entrepôt de Données de Santé** pour le **pilotage** et la **recherche**. Vous allez le bâtir : de la donnée brute déposée chaque jour... jusqu'aux dashboards.

Partie 1 — Interface d'analyse

Architecture + traitements + dashboards + dossier écrit.

C27→C30

Partie 2 — Automatisation

Pipeline planifié, robuste, tracé + documentation. **C31**

La théorie de ce matin = votre boîte à outils

Tout ce qu'on voit maintenant, vous l'appliquerez directement sur le projet.

01

Le monde de la Big Data

Les 5V · Pourquoi c'est possible maintenant

Big Data : les 5 V

V

Volume

beaucoup de données (ex. monitoring)

V

Vélocité

arrivée rapide / en continu

V

Variété

CSV, JSON, Parquet, logs...

V

Véracité

qualité, fiabilité, doublons

V

Valeur

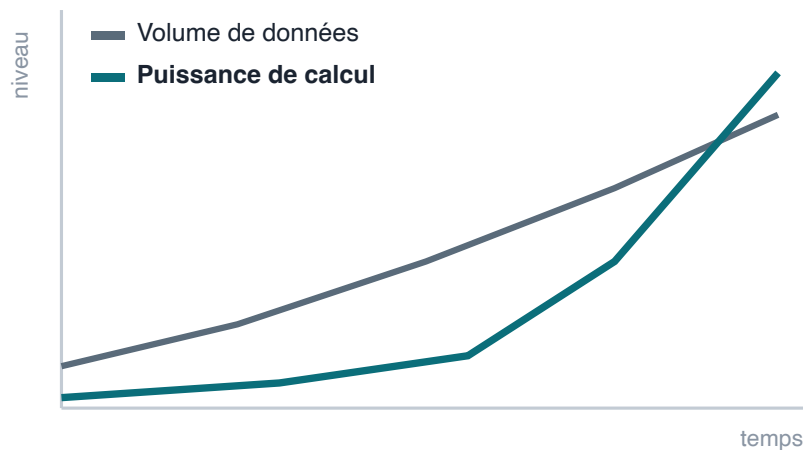
ce qu'on en tire pour décider

Dans notre projet

Le flux **monitoring** (constantes au chevet) incarne **Volume + Vélocité** ; le mélange CSV/JSON/Parquet, la **Variété** ; les anomalies, la **Véracité** ; les KPI, la **Valeur**.

1 · POURQUOI MAINTENANT ?

La donnée a toujours existé — la puissance de calcul, non



On a **toujours** accumulé de la donnée (transactions, logs, capteurs). Ce qui est **nouveau**, c'est de pouvoir l'**exploiter** à grande échelle.

Stockage bon marché

Garder **tout** ne coûte presque plus rien

Calcul distribué & cloud

De la puissance à **la demande**, parallélisée

Moteurs modernes

Bases **colonnes** pensées pour l'analytique

L'idée

Big Data ≠ « plus de données ». Big Data = « on peut **enfin en tirer de la valeur** » — le 5^e V.

02

Les métiers & la stack

Data eng / analyst / scientist · Du Postgres au warehouse

Le « starter pack » de la data



Data Engineer

Construit les **pipelines** et le **modèle de données**, gère l'ingestion et les plateformes.



Data Analyst

Analyse et **restitue** via des outils de **visualisation** (dashboards).



Data Scientist

Crée des **modèles de machine learning** (prédiction).

Data Engineer

sert la donnée



Analyst & Scientist

exploitent la donnée



Métier

décisions & stratégie

Cette semaine

Vous êtes dans le rôle du **Data Engineer** : vous **servez** une donnée fiable, prête à être exploitée.

Le « starter pack » d'une data platform



Petite boîte / début

PostgreSQL

Python

VM

Docker

Un **Postgres bien indexé** + quelques scripts suffisent longtemps.



plafond
de perf



Grande échelle

AWS / GCP / Azure

Snowflake

BigQuery

ClickHouse

Warehouse **colonne**, **compute distribué**,
partitionnement.

L'idée

On peut faire de la Big Data avec un simple **Postgres bien indexé** — beaucoup de boîtes démarrent comme ça. Quand la perf **plafonne**, on bascule vers un **warehouse** (stockage colonne + calcul distribué).

03

Structurer & transformer

ETL → ELT · Batch vs Flux · Le médaillon

3 · TRAITEMENT

ETL → ELT

Avant — ETL

Extract → Transform → Load → base

On transformait **avant** de charger : le stockage coûtait cher, on ne gardait que le résultat final.

Aujourd'hui — ELT

Extract → Load → lake → Transform ×N

Le stockage ne coûte presque rien : on **charge tout** le brut, puis on enchaîne les transformations.

Pourquoi l'ELT aujourd'hui ?

On garde la source brute

Rejouer, corriger une transfo, répondre à un nouveau besoin — sans ré-extraire.

Stockage bon marché

Plus besoin de jeter : on conserve tout l'historique.

Le warehouse calcule

Transformer en SQL, à l'échelle, là où est la donnée.

Traçabilité & conformité

On sait toujours d'où vient la donnée (utile RGPD/audit).

3 · LE PATRON MÉDAILLON

Bronze → Silver → Gold



L'idée

On **monte en valeur** à chaque couche : brut → fiable → décisionnel. Chaque couche a **un seul rôle** — c'est ce qui rend le pipeline lisible et maintenable.

Batch vs Flux, et comment on choisit

Traitement batch

Par lots, à intervalles (ex. chaque nuit). Simple, robuste.

C'est notre cas (dépôt quotidien).

Traitement en flux (stream)

En temps réel, au fil de l'eau. Puissant mais plus complexe (Lambda / Kappa).

Critères de choix

- Fréquence de la donnée (temps réel ?)
- Volume & coût
- Complexité acceptable
- Besoin métier réel

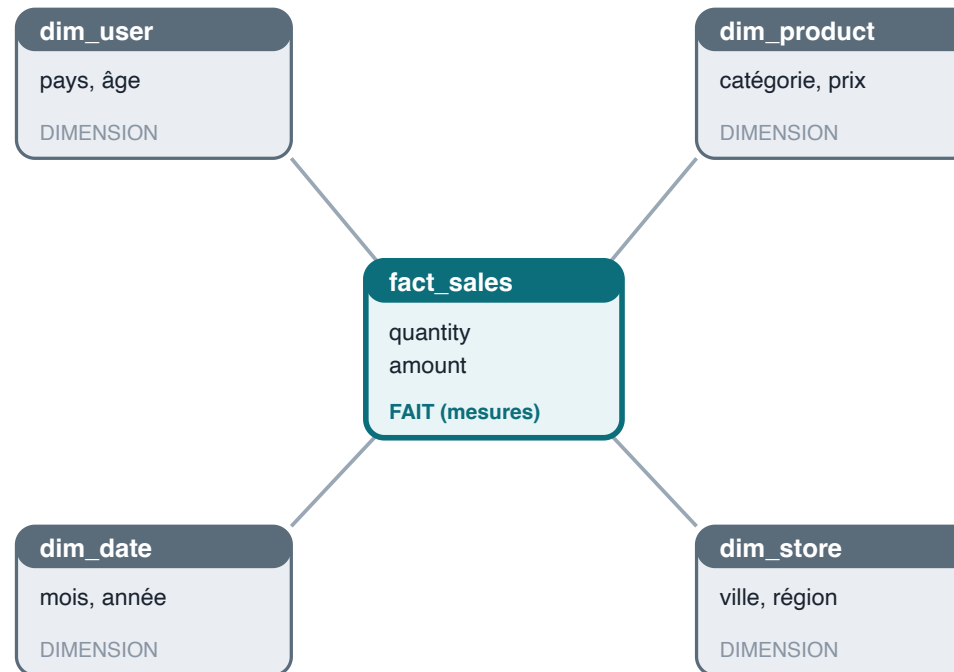
La bonne archi = celle qui répond au **besoin**, pas la plus « à la mode ».

04

Modéliser la donnée

Le schéma en étoile · Fait ou dimension ?

Le schema en etoile



Fait

un **événement mesurable** — ex. fact_sales (quantité, montant). On l'**agrège**.

Dimension

un **axe d'analyse** (le « par... ») — user, product, date, store.

Référentiel

une dimension **code** → **libellé** (nomenclature).

Fait ou dimension ? Partez du besoin

« nombre de ventes **par produit** »



Ce qu'on compte → le FAIT

« nombre de ventes » = l'**événement** mesuré → **fact_sales** (1 ligne = 1 vente). C'est ce qu'on **agrège**.



Le « par... » → la DIMENSION

« par produit » = l'**axe d'analyse** → **dim_product** (type de produit). C'est ce par quoi on **groupe**.

```
select p.type_produit, count(*) -- count = mesure (FAIT) from fact_sales s join dim_product p using (product_id) group by p.type_produit -- « par produit » = DIMENSION
```

La règle

Dans « **KPI par X** » : **X est une dimension**, et le KPI (compte / somme) sort du **fait**. ⚠ On ne crée **jamais** un « fact_produit » — le produit n'est pas un événement.

05

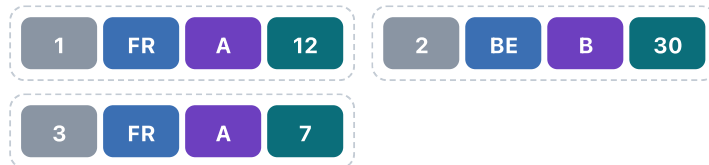
L'entrepôt

Ligne vs colonne · Partitionnement · ClickHouse

Stockage en ligne vs en colonne

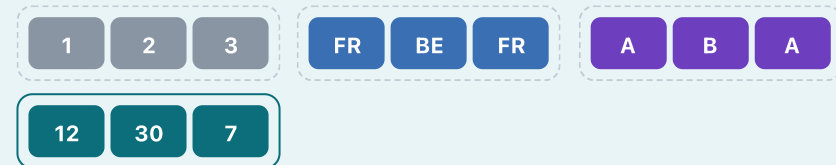
Même table (id · user · product · amount) — deux façons de la ranger sur le disque.

En LIGNE (bases transactionnelles)



Une ligne entière est contiguë → parfait pour **lire/écrire 1 enregistrement**. Mais pour `SUM(amount)`, il faut **tout lire**.

En COLONNE (entrepôts analytiques)

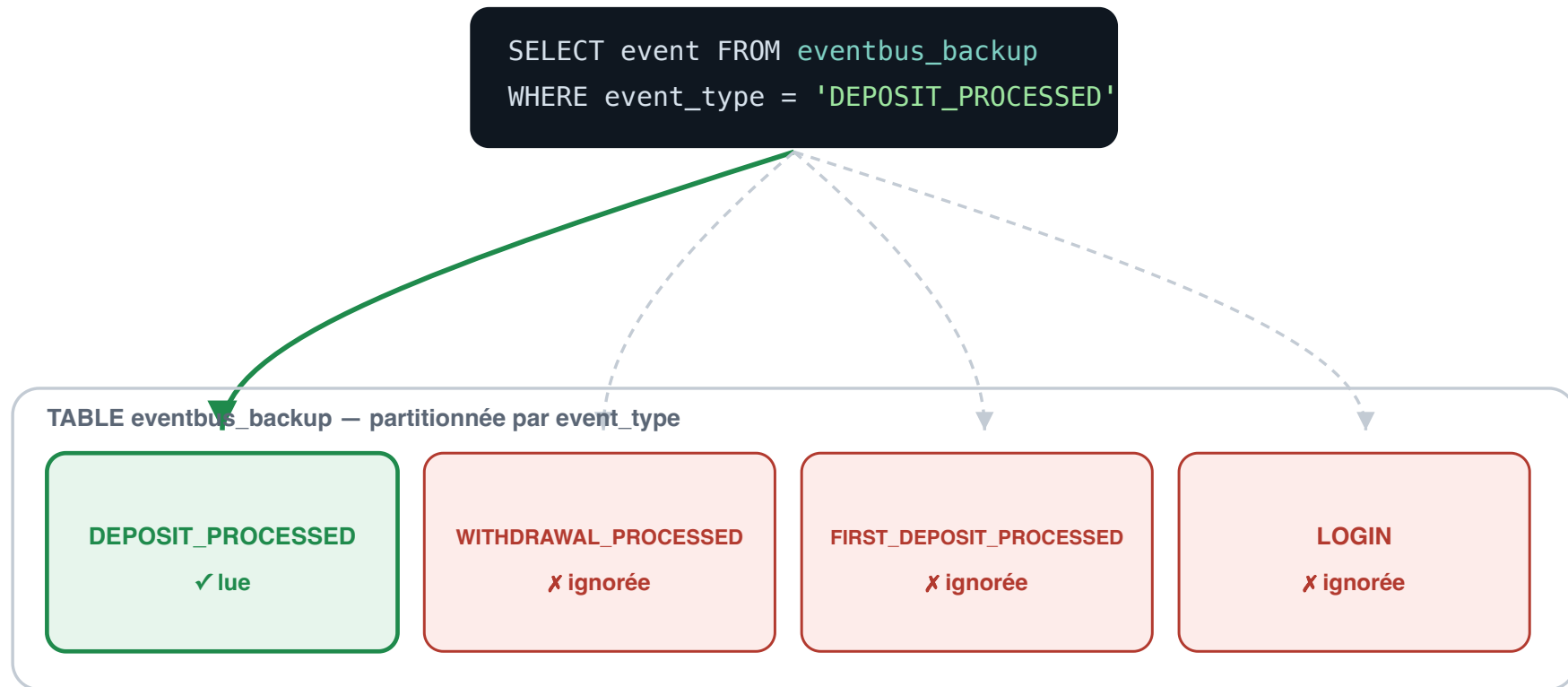


Chaque colonne est contiguë → pour `SUM(amount)`, on ne lit **que la colonne amount** (encadrée). Bonus : **forte compression**.

D'où les entrepôts colonnes

L'analytique agrège **quelques colonnes** sur **des millions de lignes** → le colonnaire ne lit que l'utile. C'est le choix de **ClickHouse, Snowflake, BigQuery**.

Le partitionnement



Partition pruning

La table est découpée par event_type : la requête ne lit **que** la partition **DEPOSIT_PROCESSED** (verte), les autres sont ignorées. Moins de données lues = requête plus rapide.

ClickHouse : un entrepôt en colonnes

- **Colonnaire** : ne lit que les colonnes utiles → analytique ultra-rapide
- **Partitionnement** : découpe la donnée pour n'en lire qu'une partie
- **Distribuable** : conçu pour passer à l'échelle (cluster)
- Une **UI SQL** intégrée (:8123/play)

Dans le projet

ClickHouse en **1 conteneur Docker**, avec sa **console SQL** (:8123/play). C'est notre entrepôt : on y construit **bronze / silver / gold** en SQL.

On garde le discours « vrai warehouse Big Data » — sans usine à gaz : tout tourne sur un laptop.

06

Gouvernance & conformité

RGPD & données de santé

RGPD & données de santé

Les données de santé sont une **catégorie particulière** (RGPD, art. 9). La conformité est une **contrainte de conception**, pas une option.

Pseudonymisation

Aucune identité en clair dans l'entrepôt (hash stable)

Minimisation

Ne garder que l'utile (date → année)

Cloisonnement

Pilotage ≠ Recherche → droits distincts

k-anonymat

Cohortes < 5 patients non diffusées

Traçabilité

D'où vient la donnée, quand traitée

Conservation

Durées limitées, finalité définie

07

Le projet & l'épreuve

Le sujet EDS du CHU · L'épreuve E05

Votre mission : l'EDS du CHU



Vos rendus



Un rapport écrit (court)

- Le **besoin** métier
- Les **choix** faits (technos, traitements) et leur justification
- L'**architecture** (schéma) + limites & recommandations



Un repo Git

- Le **code** du pipeline (ingestion → transfos)
- Les **dashboards** (ou leur export)
- Un **README** : comment lancer & rejouer

Rappel — ce que ça couvre à l'épreuve E05

Le rapport + l'archi = **Partie 1** (interface d'analyse, C27→C30) · le pipeline automatisé du repo = **Partie 2** (automatisation, C31).

ON PASSE À LA PRATIQUE

À vous de jouer

On récupère le sujet + les données, et on lance le projet.

Des questions d'abord ?